Report Progetto Finale

Daniele Di Renzo 2128730

Valerio Da Corte 2107313

Andrei Valentin Chircu 2104581

1 Obiettivo del Progetto

Il progetto si propone di realizzare una pipeline containerizzata e modulare per l'acquisizione e l'estrazione di testo pulito da sorgenti web eterogenee, impiegando parser ottimizzati per specifici domini. Il sistema espone le funzionalità core attraverso un'API REST sviluppata con FastAPI e un'interfaccia utente (Web UI) interattiva. Inoltre, include un modulo per la valutazione automatica della qualità del testo estratto, confrontandolo con un Gold Standard validato manualmente.

2 Organizzazione del Codice

L'architettura del sistema è distribuita su più container orchestrati tramite Docker Compose, garantendo una netta separazione delle responsabilità tra i diversi moduli.

Server REST (`backend/src/server.py`). Implementa gli endpoint FastAPI necessari alla gestione del ciclo di vita dei dati e delle valutazioni, tra cui: `GET/POST /parse`, `GET /domains`, `GET /gold_standard`, `GET /gold_standard_urls`, `POST /evaluate`, `POST /evaluate_judge` e `GET /full_gs_eval`.

Valutazione (`backend/src/evaluation.py`). La classe `Evaluator` gestisce il calcolo delle metriche di confronto sia a livello di token (`token_level_eval`) sia attraverso metriche avanzate (`x_eval`). La funzione `evaluate_gold_standard` coordina il processo di valutazione sull'intero Gold Standard, selezionando dinamicamente il parser idoneo per ciascun dominio. Le metriche addizionali integrate includono la Cosine Similarity, il BoilerPlate Ratio & Fidelity, la distanza di Levenshtein Normalizzata e il punteggio ROUGE-L.

Parser (`backend/src/parsers/`). La classe astratta `BaseParser` definisce la logica comune a tutti i componenti di scraping: l'esecuzione del crawling tramite `Crawl4AI`, il metodo `remove_markdown` per la conversione del formato Markdown in testo piano e la funzione `run_from_html` per il parsing di sorgenti HTML locali. Le sottoclassi specializzate (`WikipediaParser`, `IlPostParser`, `StatMuseParser` ed `EurLexParser`) estendono la classe base sovrascrivendo i metodi `create_crawler_config`, `extract_title`, `parse_markdown` e `format_result`, applicando selettori CSS e logiche di post-elaborazione modellate sulla struttura dello specifico dominio.

Frontend (`frontend/src/frontend.py`). Sviluppato anch'esso tramite un server FastAPI con motore di templating Jinja2, offre una Web UI articolata in diverse sezioni operative:

- **Home Page:** mostra lo stato operativo dei servizi (Ollama, database, backend) e i domini attualmente supportati tramite chiamate API esterne.
- **Pagina di Valutazione:** consente di sottoporre un URL al processo di parsing (con acquisizione live o da file HTML locale) e visualizza i risultati del confronto, sia per quanto riguarda le metriche statistiche standard sia per il giudizio qualitativo dell'LLM.
- **Gestione Gold Standard:** permette di interagire direttamente con la base di dati per aggiungere o rimuovere URL dal dataset di riferimento.
- **Statistiche:** raccoglie e mostra l'aggregazione dei risultati dei test eseguiti. I dati sono strettamente legati allo stato del database: in caso di ripristino o reset del DB, i contatori vengono azzerati fino all'esecuzione di nuove sessioni di test.

Gold Standard (`gs_data/`). Per ciascun dominio è strutturato un file `gs.json` contenente i metadati di 10 record (URL, dominio, titolo e path dei file). Il testo estratto di riferimento è memorizzato in file esterni all'interno delle cartelle `texts/` anziché inline nel JSON; tale approccio mantiene i file di configurazione leggeri e ottimizza il versionamento del codice. All'avvio dell'applicazione, questi dati vengono caricati automaticamente nel database relazionale.

Orchestrazione (`docker-compose.yaml`). Gestisce il deployment simultaneo di backend (porta `8003`), frontend (porta `8004`), database (porta `3306`) e Ollama (porta `11434`). L'avvio dei servizi segue vincoli di precedenza rigorosi tramite la direttiva `depends_on` con condizione `service_healthy`: il frontend viene inizializzato solo dopo il corretto avvio del backend, il quale a sua volta attende il superamento degli healthcheck del database e del server Ollama.

Database (`backend/src/db/`). Cura la persistenza dei dati attraverso un'architettura modulare: `connection.py` configura il ciclo di vita delle connessioni, `init_db.py` si occupa dell'inizializzazione dello schema e del popolamento iniziale tramite il Gold Standard, e `repository.py` implementa il pattern Repository per astrarre le operazioni CRUD. Oltre alle tabelle core obbligatorie (`web_resources` e `gold_standard`), lo schema è stato esteso con le tabelle `parsed_documents` (per gli output dei parser), `evaluations` (per le metriche automatiche) e `llm_judgements` (per i riscontri qualitativi dell'LLM).

Judge (`backend/src/ollama/`). Incapsula la logica di valutazione semantica basata sul paradigma *LLM-as-a-Judge* tramite l'integrazione con Ollama. Fornendo al modello locale il testo estratto e il rispettivo testo di riferimento del Gold Standard, il modulo interroga l'LLM per ottenere un output strutturato, composto da un punteggio numerico (da 1 a 5) e da una sintetica motivazione testuale. Per mitigare i costi computazionali dell'inferenza locale, i testi in input vengono preventivamente troncati a una soglia massima di circa 400 caratteri.

La progettazione del prompt segue criteri di ingegneria strutturata: viene definito chiaramente il contesto operativo (ruolo del modello), seguito dalle istruzioni dettagliate di analisi e dalle specifiche sul formato di output atteso. Al fine di minimizzare la lunghezza del contesto e velocizzare i tempi di risposta, si è optato per una configurazione *zero-shot*, omettendo esempi espliciti.

3 Valutazione dei Parser

3.1 Metodologia

Per ogni dominio sono stati selezionati 10 URL rappresentativi, ricavando manualmente il testo informativo (titolo e corpo, escludendo elementi strutturali come menu, footer e sidebar) per la costruzione del Gold Standard.

La metrica principale `token_level_eval` confronta i token (normalizzati in minuscolo e privati di punteggiatura) dei due testi mediante la classe `Counter`, preservando le frequenze assolute delle parole rispetto a una valutazione basata su insiemi puri. Questa scelta metodologica permette di penalizzare correttamente anomalie di parsing come la duplicazione di interi paragrafi, che altrimenti non verrebbero rilevate da un approccio a insieme (*set*).

3.2 Risultati

Dominio	Prec.	Rec.	F1
en.wikipedia.org	0.993	0.986	0.989
www.ilpost.it	0.999	1.000	1.000
www.statmuse.com	0.999	1.000	0.999
eur-lex.europa.eu	0.969	1.000	0.984

Table 1: Medie `token_level_eval` per dominio (post-ottimizzazione).

I risultati empirici evidenziano un'elevata accuratezza di estrazione nella maggior parte dei domini considerati. Il dominio `statmuse.com` ha tuttavia presentato le maggiori criticità operative, dovute alla natura dinamica delle pagine i cui dati vengono iniettati tramite script asincroni anziché risiedere nell'HTML statico. Per superare tale limite, a differenza degli altri domini gestiti via `Crawl4AI`, si è implementata una logica di scraping personalizzata basata su `BeautifulSoup`.

La strategia di estrazione è stata inoltre differenziata in base a pattern specifici rilevati negli URL (es. `scores`, `money`, `standings`). Nello specifico scenario delle pagine dedicate ai calendari e ai match programmati, il sistema è riuscito a processare correttamente solo le informazioni relative agli eventi già conclusi, determinando un calo delle performance metriche rispetto alle altre sorgenti analizzate.